REST API 설계서

문서 정보

- 프로젝트명: (SK Shieldus Rookies 내부 협업 메신저) - peerFlow
- 작성자: SK쉴더스 루키즈 4기 개발 2팀
- 작성일: 2025-09-16
- 버전: v1.0
- 작성자: 김현근
- **Base URL**: `http://localhost:8080/api`

---

1. API 설계 개요

1.1 설계 목적

RESTful 원칙에 따라 클라이언트-서버 간 통신 규격을 정의하여, SK쉴더스 루키즈 교육생 및 관계자 간의 원활한 협업(채팅, 과제 관리, 공지) 환경을 지원하는 일관되고 확장 가능한 API를 제공합니다.

1.2 설계 원칙

- **RESTful** 아키텍처: HTTP 메서드와 상태 코드의 올바른 사용
- 일관성: 모든 API 엔드포인트에서 동일한 규칙 적용 (리소스는 복수형 명사 사용)
- 버전 관리: URL 경로를 통한 버전 구분 (`/api`를 v1으로 간주)
- 보안: JWT 기반의 Stateless 인증 적용
- 성능: 페이지네이션을 통한 응답 속도 최적화
- 문서화: OpenAPI 3.0 (Swagger)을 통한 명확한 요청/응답 스펙 제공

1.3 기술 스택

- 프레임워크: Spring Boot 3.5.5 (Spring 6)
- 인증: Spring Security + JWT (JSON Web Token)
- 직렬화: JSON (Jackson)
- **API** 문서: springdoc-openapi 3.0 (Swagger)

---

2. API 공통 규칙

2.1 URL 설계 규칙

규칙	설명	좋은 예	나쁜 예
명사 사용	동사가 아닌 명사로 리소스 표현	`/api/assignments`	`/api/getAssignments`
복수형 사용	컬렉션은 복수형으로 표현	`/api/users`	`/api/user`
계층 구조	리소스 간 관계를 URL로 표현	`/api/assignments/1/submissions`	`/api/getAssignmentSubmissions`
소문자 사용	URL은 소문자와 하이픈(필요시) 사용	`/api/chat-rooms`	`/api/ChatRooms`
동작 표현	HTTP 메서드로 동작 구분	`POST /api/assignments`	`/api/createAssignment`

2.2 HTTP 메서드 사용 규칙

메서드	용도	멱등성	안전성	예시
GET	리소스 조회	✅	✅	`GET /api/announcements`
POST	리소스 생성	❌	❌	`POST /api/users/signup`
PUT	리소스 전체 수정	✅	❌	`PUT /api/assignments/1`
PATCH	리소스 부분 수정	❌	❌	`PATCH /api/users/me/nickname`
DELETE	리소스 삭제	✅	❌	`DELETE /api/assignments/1`

2.3 HTTP 상태 코드 가이드

코드	상태	설명	사용 예시

200	OK	성공 (데이터 포함)	GET, PUT, PATCH 요청 성공
201	Created	리소스 생성 성공	POST 요청 성공
204	No Content	성공 (응답 데이터 없음)	DELETE 요청 성공
400	Bad Request	잘못된 요청	JSON 문법 오류, 파라미터 누락
401	Unauthorized	인증 필요	유효한 JWT 토큰 없음
403	Forbidden	권한 없음	ADMIN 전용 API를 STUDENT가 호출
404	Not Found	리소스 없음	존재하지 않는 ID로 조회 시도
409	Conflict	충돌	중복된 이메일/닉네임으로 가입 시도
422	Unprocessable Entity	의미상 오류	비즈니스 규칙 위반 (예: 마감기한 지난 과제 제출)
500	Internal Server Error	서버 오류	처리되지 않은 예외 발생

2.4 공통 요청 헤더

Content-Type: application/json
Authorization: Bearer {JWT_TOKEN}

2.5 공통 응답 형식

peerFlow 프로젝트는 성공 응답 시 Wrapper 객체 없이 **DTO** 또는 **`Page<DTO>`** 객체를 직접 **Body**로 반환합니다.

성공 응답 **(**단일 객체 **-** 예**: `GET /api/users/me`)**

JSON
```
{
  "id": 10,
  "username": "김학생",
  "nickname": "열공모드",
  "email": "student@example.com",
  "role": "STUDENT",
  "status": "ACTIVE",
  "avatarUrl": null
```

```
}
```

성공 응답 (목록/페이지네이션 - 예: `GET /api/announcements`)

Spring Data JPA의 `Page` 객체가 직렬화된 표준 형식입니다.

JSON
```
{
  "content": [
    {
      "id": 2,
      "title": "중요 공지",
      "authorNickname": "관리자",
      "isPinned": true,
      "createdAt": "2025-09-16T10:00:00Z"
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
     // ... 기타 페이징 정보
  },
  "totalPages": 1,
  "totalElements": 1,
  "last": true,
  // ...
}
```

에러 응답 (커스텀 `BusinessException` 예외 기준)

JSON
```
{
  "message": "User(을)를 찾을 수 없습니다. (email: wrong@email.com)",
  "statusCode": "404"
}
```

---

3. 인증 및 권한 관리

3.1 JWT 토큰 기반 인증

3.1.1 회원가입 API

POST /api/users/signup
Content-Type: application/json

Request Body:
```
{
  "email": "student@example.com",
  "password": "password123!",
  "username": "김학생",
  "nickname": "열공모드",
  "role": "STUDENT"
}
```

Response (200 OK):
(사용자 DTO 반환, 상태는 PENDING)

3.1.2 로그인 API

POST /api/auth/login
Content-Type: application/json

Request Body:
```
{
  "email": "student@example.com",
  "password": "password123!"
}
```

Response (200 OK):
(계정이 ACTIVE 상태일 때만 성공)
```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9..."
}
```

Response (401 Unauthorized):
{ "message": "Invalid credentials", "statusCode": "401" }

3.1.3 비밀번호 초기화 API

1. 인증번호 발송
POST /api/auth/password/reset
Query Parameters:
- email: string (required)
Response (200 OK): "인증번호가 발송되었습니다."

2. 인증번호 검증
POST /api/auth/password/verify
Request Body:
```
{
  "email": "user@example.com",
  "code": "123456"
}
```
Response (200 OK): "인증번호 확인 완료"

3. 새 비밀번호 저장
POST /api/auth/password/update
Request Body:
```
{
  "email": "user@example.com",
  "code": "123456",
  "newPassword": "newPassword123!"
}
```
Response (200 OK): "비밀번호가 변경되었습니다."

3.2 권한 레벨 정의

역할	영문	설명
학생	STUDENT	과제 제출, 채팅, 내 정보 관리, 공지/과제 조회, 알림 확인
강사	TEACHER	STUDENT 권한 + 과제 생성/채점, 공지사항 관리 (CRUD)
관리자	ADMIN	회원 가입 승인/거절, 전체 사용자 관리 (삭제, 비번 초기화 등)

---

4. 상세 API 명세

4.1 인증 API (Auth Controller)

4.1.1 로그인
POST /api/auth/login
(상세 내용은 3.1.1 참고)

4.1.2 비밀번호 재설정 인증번호 발송
POST /api/auth/password/reset
Query Parameters:
- email: string (required) - 인증번호를 받을 이메일
Response (200 OK)

4.1.3 비밀번호 재설정 코드 검증
POST /api/auth/password/verify
Request Body:
```
{
  "email": "user@example.com",
```

```
  "code": "123456"
}
```
Response (200 OK)

4.1.4 새 비밀번호로 업데이트

POST /api/auth/password/update
Request Body:
```
{
  "email": "user@example.com",
  "code": "123456",
  "newPassword": "newPassword123!"
}
```
Response (200 OK)

4.2 사용자 API (User Controller)

4.2.1 회원가입

POST /api/users/signup
Content-Type: application/json

Request Body:
```
{
  "email": "student@example.com",
  "password": "password123!",
  "username": "김학생",
  "nickname": "열공모드",
  "role": "STUDENT"
}
```
Response (200 OK):
(UserDTO.Response 반환, 상태는 PENDING)

Response (409 Conflict):
{ "message": "리소스가 이미 존재합니다. (User, email, ...)" }

4.2.2 전체 활성 사용자 조회

GET /api/users/all
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Response (200 OK): List<UserResponseDTO>

4.2.3 이메일/이름으로 사용자 검색

GET /api/users/findUser
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Query Parameters:

- keyword: string (required) - 검색할 키워드
Response (200 OK): List<UserResponseDTO>

4.2.4 내 정보 조회
GET /api/users/me
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Response (200 OK): UserDTO.Response

4.2.5 내 사용자 이름 수정
PUT /api/users/me/username
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Request Body:
```
{
  "username": "김개발"
}
```
Response (200 OK): UserDTO.Response (업데이트된 사용자)

4.2.6 내 닉네임 수정
PUT /api/users/me/nickname
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Request Body:
```
{
  "nickname": "열정폭발"
}
```
Response (200 OK): UserDTO.Response (업데이트된 사용자)

4.2.7 회원 탈퇴 **(**본인**)**
DELETE /api/users/me
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Response (204 No Content)

4.2.8 내 비밀번호 변경
POST /api/users/me/change-password
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Query Parameters:
- oldPassword: string (required)
- newPassword: string (required)
Response (200 OK): UserDTO.Response

4.2.9 프로필 이미지 업로드

POST /api/users/{id}/avatar
Content-Type: multipart/form-data
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN (본인 또는 ADMIN)
Path Parameters:
- id: integer (required) - 사용자 ID
Response (200 OK): UserDTO.Response (avatarUrl이 업데이트된 사용자)

4.2.10 프로필 이미지 삭제

DELETE /api/users/{id}/avatar/{fileName}
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN (본인 또는 ADMIN)
Path Parameters:
- id: integer (required) - 사용자 ID
- fileName: string (required) - 삭제할 파일명
Response (200 OK): UserDTO.Response (avatarUrl이 null로 업데이트된 사용자)

4.3 관리자 **API (Admin Controller)**

`Required Role: ADMIN` (모든 API)

4.3.1 전체 사용자 목록 조회 **(**페이징**)**

GET /api/admin/users
Authorization: Bearer {JWT_TOKEN}
Query Parameters:
- pageable: (page, size, sort)
Response (200 OK): Page<UserDTO.Response>

4.3.2 상태별 사용자 목록 조회

GET /api/admin/users/status/{status}
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- status: string (required) - (PENDING, ACTIVE, REJECTED 등)
Query Parameters:
- pageable: (page, size, sort)
Response (200 OK): Page<UserDTO.Response>

4.3.3 역할별 사용자 목록 조회

GET /api/admin/users/role/{role}
Authorization: Bearer {JWT_TOKEN}
Path Parameters:

- role: string (required) - (STUDENT, TEACHER, ADMIN)
Query Parameters:
- pageable: (page, size, sort)
Response (200 OK): Page<UserDTO.Response>

4.3.4 사용자 가입 승인
PATCH /api/admin/users/{id}/approve
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- id: integer (required) - 승인할 사용자 ID
Response (200 OK): UserDTO.Response

4.3.5 사용자 가입 거절
PATCH /api/admin/users/{id}/reject
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- id: integer (required) - 거절할 사용자 ID
Response (200 OK): UserDTO.Response

4.3.6 사용자 비활성화
PATCH /api/admin/users/{id}/deactivate
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- id: integer (required) - 비활성화할 사용자 ID
Response (200 OK): UserDTO.Response

4.3.7 사용자 정보 수정 **(**관리자용**)**
PUT /api/admin/users/{id}
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- id: integer (required) - 수정할 사용자 ID
Request Body:
(UserDTO.Request - username, nickname, role, status 등 수정 가능)
Response (200 OK)

4.3.8 사용자 비밀번호 강제 초기화
POST /api/admin/users/{id}/reset-password
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- id: integer (required) - 초기화할 사용자 ID
Query Parameters:
- newPassword: string (required) - 관리자가 설정할 새 비밀번호
Response (200 OK)

4.3.9 사용자 삭제 (관리자용)

DELETE /api/admin/users/{id}
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- id: integer (required) - 삭제할 사용자 ID
Response (204 No Content)

4.4 공지사항 API (Announcement Controller)

4.4.1 공지사항 목록 조회

GET /api/announcements
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Query Parameters:
- pageable: (page, size) - (정렬은 isPinned(desc), createdAt(desc)로 고정)
Response (200 OK): Page<AnnouncementDTO.Response>

4.4.2 공지사항 상세 조회

GET /api/announcements/{id}
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Path Parameters:
- id: integer (required) - 조회할 공지사항 ID
Response (200 OK): AnnouncementDTO.Response

4.4.3 공지사항 생성

POST /api/announcements
Authorization: Bearer {JWT_TOKEN}
Required Role: TEACHER, ADMIN
Request Body:
```
{
  "title": "중요 공지입니다",
  "content": "내용...",
  "isPinned": true
}
```
Response (201 Created): AnnouncementDTO.Response

4.4.4 공지사항 수정

PUT /api/announcements/{id}
Authorization: Bearer {JWT_TOKEN}
Required Role: TEACHER (본인), ADMIN
Path Parameters:

- id: integer (required) - 수정할 공지사항 ID
Request Body:
```
{
  "title": "수정된 제목",
  "content": "수정된 내용...",
  "isPinned": false
}
```
Response (200 OK): AnnouncementDTO.Response

4.4.5 공지사항 삭제

DELETE /api/announcements/{id}
Authorization: Bearer {JWT_TOKEN}
Required Role: TEACHER (본인), ADMIN
Path Parameters:
- id: integer (required) - 삭제할 공지사항 ID
Response (204 No Content)

4.5 과제 **API (Assignment Controller)**

4.5.1 과제 목록 조회

GET /api/assignments
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN
Response (200 OK): List<AssignmentPreviewResponseDTO>

4.5.2 새 과제 생성

POST /api/assignments
Authorization: Bearer {JWT_TOKEN}
Required Role: TEACHER, ADMIN
Request Body:
```
{
  "title": "1주차 REST API 설계 과제",
  "content": "설계서를 작성하여 제출하세요.",
  "dueDate": "2025-09-30T23:59:59"
}
```
Response (201 Created)

4.5.3 과제 정보 수정

PATCH /api/assignments/{assignmentId}
Authorization: Bearer {JWT_TOKEN}
Required Role: TEACHER (본인), ADMIN
Request Body: AssignmentUpdateRequestDTO
Response (200 OK)

4.5.4 과제 삭제

DELETE /api/assignments/{assignmentId}
Authorization: Bearer {JWT_TOKEN}
Required Role: TEACHER (본인), ADMIN
Path Parameters:
- assignmentId: integer (required)
Response (204 No Content)

4.5.5 과제 제출

POST /api/assignments/{assignmentId}/submissions
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT
Path Parameters:
- assignmentId: integer (required) - 과제 ID
Request Body:
```
{
  "submissionContent": "과제 제출합니다."
}
```
Response (200 OK)

4.5.6 제출 과제 채점

PATCH /api/submissions/{submissionId}
Authorization: Bearer {JWT_TOKEN}
Required Role: TEACHER, ADMIN
Path Parameters:
- submissionId: integer (required) - 제출물 ID
Request Body:
```
{
  "score": 95,
  "feedback": "설계가 훌륭합니다."
}
```
Response (200 OK)

4.6 채팅방 API (Chat Room Controller)

`Required Role: STUDENT, TEACHER, ADMIN` (모든 API)

4.6.1 새 채팅방 생성

POST /api/chatrooms
Authorization: Bearer {JWT_TOKEN}
Content-Type: application/json
Request Body: CreateRoomRequestDTO
```
{
  "roomName": "새 그룹채팅방",
```

```
  "type": "GROUP",
  "participantIds": [2, 3]
}
```
Response (200 OK): ChatRoomResponseDTO

4.6.2 채팅방에 사용자 초대

POST /api/chatrooms/{roomId}/participants/{userId}
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- roomId: integer (required) - 초대할 채팅방 ID
- userId: integer (required) - 초대할 사용자 ID
Response (200 OK)

4.6.3 채팅방 입장

POST /api/chatrooms/{roomId}/join
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- roomId: integer (required) - 입장할 채팅방 ID
Response (200 OK)

4.6.4 채팅방 나가기

PUT /api/chatrooms/{roomId}/leave
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- roomId: integer (required) - 나갈 채팅방 ID
Response (200 OK)

4.6.5 채팅방 참여자 목록 조회

GET /api/chatrooms/{roomId}/participants
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- roomId: integer (required) - 채팅방 ID
Response (200 OK): List<UserDTO.Response>

4.6.6 채팅방 이전 메시지 목록 조회

GET /api/chatrooms/{roomId}/messages
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- roomId: integer (required) - 채팅방 ID
Response (200 OK): List<ChatMessageDTO>

4.6.7 내 모든 채팅방 목록 조회

GET /api/chatrooms/all
Authorization: Bearer {JWT_TOKEN}
Response (200 OK): List<ChatRoomResponseDTO> (읽지 않은 메시지 수 포함)

4.6.8 채팅방 알림 켜기/끄기

PATCH /api/chatrooms/{roomId}/mute
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- roomId: integer (required) - 채팅방 ID
Response (200 OK)

4.6.9 채팅방 상단 고정/해제

PATCH /api/chatrooms/{roomId}/pin
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- roomId: integer (required) - 채팅방 ID
Response (200 OK)

4.7 알림 **API (History Controller)**

`Required Role: STUDENT, TEACHER, ADMIN` (모든 API)

4.7.1 내 알림 목록 조회

GET /api/histories
Authorization: Bearer {JWT_TOKEN}
Response (200 OK): List<HistoryResponseDTO>
(상세 내용은 2.5 성공 응답 (단일 객체) 예시와 유사)

4.7.2 알림 읽음 처리

PATCH /api/histories/{historyId}/read
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- historyId: integer (required) - 읽음 처리할 알림 ID
Response (200 OK)

4.7.3 알림 삭제

DELETE /api/histories/{historyId}
Authorization: Bearer {JWT_TOKEN}
Path Parameters:
- historyId: integer (required) - 삭제할 알림 ID
Response (204 No Content)

4.8 파일 API (File Controller)

4.8.1 파일 업로드

POST /api/files/upload
Content-Type: multipart/form-data
Authorization: Bearer {JWT_TOKEN}
Required Role: STUDENT, TEACHER, ADMIN

Response (200 OK):
```
{
  "fileUrl": "/uploads/avatars/user_123_timestamp.png"
}
```

4.8.2 파일 다운로드

POST /api/files/download/{fileName}
Path Parameters:
- fileName: string (required) - 다운로드할 파일의 전체 이름
Response (200 OK): (Resource Stream)
(Content-Disposition 헤더로 인해 파일 다운로드 실행됨)

4.9 채팅 메시지 API (WebSocket)

(STOMP 프로토콜 사용)

4.9.1 메시지 발신

Send Destination: STOMP /app/chat/rooms/{roomId}
(클라이언트가 서버로 메시지를 보낼 때 사용하는 경로)
Payload: SendMessageRequestDTO
```
{
  "receiverId": "수신자ID" (1:1 채팅 시),
  "content": "메시지 내용",
  "fileUrl": "파일 URL" (이미지/파일 전송 시),
  "type": "TEXT" (TEXT, IMAGE, FILE)
}
```

4.9.2 메시지 읽음 처리

Send Destination: STOMP /app/chat/rooms/{roomId}/read
(클라이언트가 메시지 읽었음을 서버에 알리는 경로)
Payload: ReadMessageRequestDTO
```
{
  "lastMessageId": 123
}
```

4.9.3 메시지 수신

Subscription Topic 1: /topic/chat/rooms/{roomId}
- (그룹 채팅방 메시지 수신용)

Subscription Topic 2: /queue/messages/{roomId}
- (1:1 채팅 메시지 수신용)

5.1 표준 에러 코드 정의

코드 (Enum)	HTTP 상태	설명	해결 방법
INVALID_INPUT_VALUE	400	입력값 검증 실패	요청 데이터 포맷 확인 후 재시도
INVALID_CREDENTIALS	401	인증 정보 오류 (로그인 실패)	이메일 또는 비밀번호 확인
TOKEN_EXPIRED	401	토큰 만료	토큰 갱신 또는 재로그인
ACCESS_DENIED	403	권한 없음	요청 권한 확인 또는 관리자 문의
RESOURCE_NOT_FOUND	404	리소스를 찾을 수 없음	요청 URL 및 리소스 ID 확인
RESOURCE_DUPLICATE	409	중복 생성 시도 (자원 충돌)	입력 값(이메일, 닉네임 등) 확인
RATE_LIMIT_EXCEEDED	429	요청 한도 초과	잠시 후 재시도
INTERNAL_SERVER_ERROR	500	서버 내부 오류	서버 로그 확인 및 관리자 문의

5.2 비즈니스 로직 에러 코드

코드 (개념)	설명	해결 방법
USER_NOT_FOUND	회원을 찾을 수 없음	회원 ID 또는 이메일 확인
ASSIGNMENT_NOT_FOUND	과제를 찾을 수 없음	과제 ID 확인
SUBMISSION_NOT_FOUND	제출물을 찾을 수 없음	제출물 ID 확인
HISTORY_NOT_FOUND	알림을 찾을 수 없음	알림 ID 확인
DUPLICATE_EMAIL	이미 사용 중인 이메일	다른 이메일로 가입 시도
DUPLICATE_NICKNAME	이미 사용 중인 닉네임	다른 닉네임으로 변경 시도
PASSWORD_SAME_AS_CURRENT	현재 비밀번호와 동일한 새 비밀번호	다른 비밀번호를 입력
INVALID_PASSWORD	현재 비밀번호 불일치	올바른 현재 비밀번호를 입력
SUBMISSION_OVERDUE	과제 제출 기한 마감	마감 기한 확인
AI_BLOCKED	AI 보안 시스템에 의해 차단됨	요청 내용 확인 또는 관리자 문의

---

6. API 문서화

6.1 OpenAPI 3.0 스펙 예시 (Swagger Config)

Java
```
// Spring Boot + springdoc-openapi 설정
@Configuration
public class SwaggerConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("peerFlow API")
                        .version("v1.0.0")
```

```
            .description("SK쉴더스 루키즈 협업 메신저 peerFlow REST API"))
        .addSecurityItem(new SecurityRequirement().addList("BearerAuth"))
        .components(new Components()
            .addSecuritySchemes("BearerAuth",
                new SecurityScheme()
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT")));
    }
}
```

6.2 Controller 어노테이션 예시

Java

```
@RestController
@RequestMapping("/api/announcements")
@Tag(name = "Announcement API", description = "공지사항 관리 API")
public class AnnouncementController {

    @GetMapping
    @Operation(summary = "공지사항 목록 조회", description = "페이지네이션된 공지사항 목록을 조회합니다")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "성공"),
        @ApiResponse(responseCode = "401", description = "인증 필요")
    })
    public ResponseEntity<Page<AnnouncementDTO.Response>> getAnnouncements(Pageable pageable) {
        // 구현 코드
    }
}
```

---

7. 체크리스트 및 품질 관리

7.1 API 설계 체크리스트

- ✅ RESTful 원칙을 준수하는가?
- ✅ URL 네이밍 규칙이 일관되는가?

- ✅ HTTP 메서드를 올바르게 사용하는가?
- ✅ HTTP 상태 코드를 적절히 사용하는가?
- ✅ 요청/응답 형식이 표준화되어 있는가?
- ✅ 에러 응답이 명확하고 도움이 되는가?
- ✅ 페이지네이션이 구현되어 있는가?
- ✅ 적절한 인증/인가가 구현되어 있는가?

7.2 보안 체크리스트

- ✅ 입력값 검증이 충분한가?
- ✅ JWT 토큰이 안전하게 관리되는가?
- ✅ 권한 체크가 모든 엔드포인트에 적용되는가?
- ✅ 에러 메시지에서 시스템 정보가 노출되지 않는가?

7.3 성능 체크리스트

- ✅ N+1 쿼리 문제가 해결되었는가? (LAZY 로딩 기본 적용)
- ✅ 데이터베이스 인덱스가 적절한가? (이메일, 닉네임 등)

---

8. 마무리

8.1 주요 포인트 요약

1. **RESTful** 설계: HTTP 메서드와 상태 코드의 올바른 사용
2. 일관성 유지: 모든 API에서 동일한 규칙과 형식 적용
3. 보안 강화: 인증, 인가, 입력 검증을 통한 보안 확보
4. 문서화: 명확하고 완전한 API 문서 제공

8.2 추천 도구 및 라이브러리

- 문서화: Swagger/OpenAPI (springdoc-openapi), Postman
- 보안: Spring Security, JWT (jjwt)

8.3 향후 고도화 방안

- 성능: 캐싱(Redis, Caffeine)을 통한 성능 향상
- 테스트: REST Assured, TestContainers
- 모니터링: 로깅, 메트릭을 통한 운영 상황 파악 (Actuator, Prometheus, Grafana)